

Deployment & Setup Guide

Local setup, production requirements, release sequence, and verification checklist

Prepared 7 October 2026 | Repository review | Branch: sartaj

Purpose and deployment status

This guide describes local setup and a provider-neutral production release for the repository as reviewed on 7 October 2026. The hosting provider is not specified, and this repository does not include a production WSGI server dependency. Add and pin the server required by the selected host before deployment. No deployment URL or live production environment is claimed here.

1. Local setup on Windows PowerShell

Install Python 3.11 or newer supported by Django 5.2 and Git. From the repository root, run:

```
py -3.11 -m venv .env
.\env\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

Open `http://127.0.0.1:8000/`. Local defaults use SQLite. The `.env.example` file is a reference; Django does not automatically load it. Optional values can be set in the shell or a secrets manager. For example, Razorpay test keys use `$env:RAZORPAY_KEY_ID`, `$env:RAZORPAY_KEY_SECRET`, and `$env:RAZORPAY_WEBHOOK_SECRET`.

2. What production services are needed

Service	Purpose / notes
Web app + domain	Run <code>footwear.wsgi:application</code> behind the hosting provider's production server and HTTPS proxy. Add and pin a compatible WSGI server in requirements if the host requires it.
PostgreSQL	Required in production through <code>DJANGO_DATABASE_URL</code> ; production settings require PostgreSQL and request SSL when not specified.
Redis	Required broker through <code>CELERY_BROKER_URL</code> in production; run a Celery worker and beat scheduler as managed processes.
S3-compatible object storage	Production media default is S3; set bucket and region plus provider credentials/endpoint where needed. Product uploads need durable storage.
Static files	Run <code>collectstatic</code> ; configure the host/CDN/static service to serve collected static assets.
Secrets and HTTPS	Use host secret settings for Django key, DB/broker credentials, provider secrets, exact hosts, CSRF origins, and TLS.
Operations	Enable DB/media backups, tested restore, logs, health monitoring, error/job alerts, and a support owner.

3. Environment variables

Variable	Required when	Purpose
<code>DJANGO_ENV=production</code>	Production	Activates production security and required-setting checks.
<code>DJANGO_SECRET_KEY</code>	Always in production	Long random secret; never commit it.
<code>DJANGO_ALLOWED_HOSTS</code>	Production	Comma-separated exact hostnames.
<code>DJANGO_CSRF_TRUSTED_ORIGINS</code>	Production/browser POSTs	HTTPS origins such as <code>https://shop.example.com</code> .

Variable	Required when	Purpose
`DJANGO_DATABASE_URL`	Production	Managed PostgreSQL URL; protect its credentials.
`CELERY_BROKER_URL`	Production	Redis broker URL.
`DJANGO_MEDIA_STORAGE`	Production default	Selects durable media storage.
`AWS_STORAGE_BUCKET_NAME`, `AWS_S3_REGION_NAME`	Production media	S3 bucket and region; set provider credentials/endpoint as required.
`RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`, `RAZORPAY_WEBHOOK_SECRET`	Online payment	API credentials and separate webhook signing secret.
`WHATSAPP_GRAPH_API_VERSION`, `WHATSAPP_PHONE_NUMBER_ID`, `WHATSAPP_ACCESS_TOKEN`, `WHATSAPP_ORDER_TEMPLATE`	WhatsApp notifications	Meta Cloud API setup; use approved templates and customer opt-in.
`FLIPKART_CLIENT_ID`, `FLIPKART_CLIENT_SECRET`, `FLIPKART_REDIRECT_URL`, `MARKETPLACE_TOKEN`, `MARKETPLACE_ENCRYPTION_KEY`	Flipkart seller authorization	Registered partner app and stable encryption key.
`AMAZON_APPLICATION`, `AMAZON_LWA_CLIENT_ID`, `AMAZON_LWA_CLIENT_SECRET`, `AMAZON_REDIRECT_URL`, `MARKETPLACE_TOKEN`, `MARKETPLACE_ENCRYPTION_KEY`	Amazon SP-API	Registered application, seller consent, roles, and region endpoint.
`ONDC_PARTICIPANT_ADAPTER`	Optional ONDC check	Dotted import path for the participant-specific adapter; credentials stay in a secret manager.

4. Release sequence

- Provision PostgreSQL, Redis, media storage, app service, HTTPS domain, and secret settings.
- Install the pinned requirements plus the chosen production WSGI/ASGI server dependency.
- Run `python manage.py check --deploy` using production environment settings; fix every configuration issue.
- Run `python manage.py showmigrations`, then apply `python manage.py migrate` as a release step.
- Run `python manage.py collectstatic --noinput` and verify static assets and uploaded media from the public app.
- Start the web process, Celery worker, and Celery beat scheduler. The WSGI target is `footwear.wsgi:application`.
- Verify `/health/`, login, seller/staff permissions, catalog images, checkout, and logs before opening the site.

5. Verify business-critical paths in staging

Area	Rehearsal
Checkout and stock	COD, variant stock changes, delivery PIN coverage/slot capacity, multi-shop split, and concurrent attempts on the last unit.

Area	Rehearsal
Razorpay	Test success, failure, bad signature, duplicate event, amount mismatch, delayed callback, refund queue, and manual-review case using test credentials.
Seller operations	Shop approval, role/data boundaries, CSV import validation, fulfillment route, proof of delivery, return/exchange, commission, and payout record.
External providers	Use approved sandbox/test accounts first. Confirm callbacks, token encryption-key backup, API roles, provider rate limits, opt-in and consent behavior.
Recovery	Take database/media backups and restore to a separate staging environment; record elapsed time and missing steps.

6. Background processes and operations

celery -A footwear worker -l INFO

celery -A footwear beat -l INFO

Run these as separate supervised services, not manually in a one-off shell. Configure process restart, logs, alerting, and a controlled release procedure. Read

[docs/production-operations.md](../docs/production-operations.md) for backup and operational notes.

7. Launch checklist

- Production-only secret values are set in the hosting platform; no keys are in Git or the document.
- `DEBUG` is false, allowed hosts and CSRF origins are exact, TLS/secure cookies are active, and `check --deploy` passes.
- PostgreSQL migration, backup, and restore are verified; media upload and retrieval survive a redeploy.
- Production server and static serving are configured; Celery processes are supervised if tasks are enabled.
- Payment webhook URL and secret are configured and rehearsed; staff know how to handle mismatches and uncertain refunds.
- Seller, support, delivery, return, payout, privacy, and incident processes have named owners for the Jaunpur pilot.
- Provider approvals and account authorizations are complete for every enabled integration; unavailable integrations remain disabled.

References

- Django 5.2 deployment checklist
- Django static files deployment
- Celery with Django
- Repository production operations notes

Before production: review the current official documentation for the selected host, provider APIs, and service versions; platform-specific networking, server command, secrets, and static hosting vary by provider.

Reservation expiry update | 8 October 2026

Razorpay checkout holds inventory for a limited payment window. The timeout is 15 minutes by default and is configurable with `ORDER_STOCK_RESERVATION_MINUTES` (minimum 1).

What the new workflow does

At checkout, the order stores its stock reservation deadline. The customer sees the deadline on the Razorpay payment page.

Celery Beat scans expired orders once per minute. Expired unpaid orders are cancelled, active payment attempts are closed, and stock is restored once.

The cancellation is recorded in the order timeline. The customer's cart remains available for a new order.

Expiry and payment callbacks lock the payment attempt before the order and recheck the current state inside a transaction. This prevents a callback that has already completed from being cancelled a second time.

Deployment requirements

Apply migration `0016_order_stock_reservation_expiry.py` with the normal release migration step: `python manage.py migrate`.

Keep Celery Beat running in production alongside the Celery worker and Redis broker. If Beat is stopped, automatic expiry runs when it resumes; expired orders may keep stock held until then.

Set `ORDER_STOCK_RESERVATION_MINUTES` in the hosting environment only when changing the 15-minute default. The setting must be a positive integer.

Late captures

A payment reported after stock has been released must not confirm the order because the item may have been sold again. Configure the Razorpay webhook and review late-capture events in staff operations before responding to the customer.